

GenerativeGUI: Dynamic GUI Generation Leveraging LLMs for Enhanced User Interaction on Chat Interfaces

Nobukatsu Hojo

Human Informatics Laboratories
Nippon Telegraph and Telephone Corporation
Yokosuka, Kanagawa, Japan
nobukatsu.hojo@ntt.com

Kazutoshi Shinoda

Human Informatics Laboratories
Nippon Telegraph and Telephone Corporation
Yokosuka, Japan
kazutoshi.shinoda@ntt.com

Yoshihiro Yamazaki

Human Informatics Laboratories
Nippon Telegraph and Telephone Corporation
Yokosuka, Japan
yoshihiro.yamazaki@ntt.com

Keita Suzuki

Human Informatics Laboratories
Nippon Telegraph and Telephone Corporation
Yokosuka, Japan
keitax.suzuki@ntt.com

Hiroaki Sugiyama

Human Informatics Laboratories
Nippon Telegraph and Telephone Corporation
Yokosuka, Japan
h.sugi@ieee.org

Kyosuke Nishida

Human Informatics Laboratories
Nippon Telegraph and Telephone Corporation
Yokosuka, Japan
kyosuke.nishida@ntt.com

Kuniko Saito

Human Informatics Laboratories
Nippon Telegraph and Telephone Corporation
Yokosuka, Japan
kuniko.saito@ntt.com

Abstract

Conversational AI (CAI) systems using large language models (LLMs) are becoming increasingly adept at performing a wide range of tasks with text-based interfaces. However, tasks that require detailed input can reduce usability and efficiency, as they often involve lengthy text exchanges. This paper introduces **GenerativeGUI**, a novel approach where LLMs dynamically generate graphical user interfaces (GUIs) tailored to the ongoing conversation. By generating HTML, which is rendered into an interactive GUI, GenerativeGUI presents intuitive widgets, enhancing user interaction and reducing cognitive load. A user study comparing GenerativeGUI with conventional text-based CAI systems demonstrated that the new system significantly improves usability, particularly in scenarios requiring multiple clarifying questions. Specifically, GenerativeGUI reduces mental demand, effort and task completion time, and enhances user satisfaction. This research highlights the potential of dynamic GUI generation to streamline complex CAI interactions, providing an effective solution to improve user experience in multi-turn conversations.

CCS Concepts

• **Information systems** → **Users and interactive retrieval**; • **Human-centered computing** → *Human computer interaction (HCI)*.

Keywords

Large Language Model, Natural Language Interface, Graphical User Interface, Clarifying Questions

ACM Reference Format:

Nobukatsu Hojo, Kazutoshi Shinoda, Yoshihiro Yamazaki, Keita Suzuki, Hiroaki Sugiyama, Kyosuke Nishida, and Kuniko Saito. 2025. GenerativeGUI: Dynamic GUI Generation Leveraging LLMs for Enhanced User Interaction on Chat Interfaces. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3706599.3719743>

1 Introduction

Recent conversational AI (CAI) systems using large language models (LLMs) can handle diverse tasks via a text-based conversational user interface (CUI). However, for complex tasks, such as conversational recommendation, travel planning, and software coding support, users often must read and write extensive text to specify requirements, understand outputs, and provide corrections, which can degrade usability, particularly in terms of interaction efficiency [17, 20, 31].

To improve interaction efficiency, previous studies have proposed combining text-based CUIs with graphical user interfaces (GUIs) [15, 20, 25]. For instance, GPT-4o with Canvas¹ enables users to directly edit documents and code on a canvas, eliminating the need to re-enter them in the prompt. Graphologue [15] converts long and complex LLM responses into graphical diagrams to assist information exploration and question-answering. While these systems enhance interaction efficiency by using a pre-designed GUI, they are limited in their ability to support the diverse tasks handled by general-purpose CAI systems powered by LLMs. This is because GUIs must be designed for specific interactions to ensure their visual elements are intuitive and easily understood by users. When a fixed, pre-designed GUI is applied to all interactions, it becomes less adaptable to the variety of tasks such systems may encounter.

To address this challenge, we need a dynamic system that can generate a GUI tailored to the context of ongoing interactions. This study proposes GenerativeGUI, a novel approach in which

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CHI EA '25, Yokohama, Japan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1395-8/25/04
<https://doi.org/10.1145/3706599.3719743>

¹<https://openai.com/index/introducing-canvas/>

Recommend a smartphone.

- What is your budget for a new smartphone?
- Do you prefer iOS or Android operating systems?
- What features are most important to you (camera, battery, etc.)?
- Are you looking for a specific brand?
- How often do you upgrade your smartphone?

Budget is \$700, prefer Android, battery is important, no specific brand and upgrade every 3 years.

You have a budget of \$700, prefer Android, prioritize battery life, have no specific brand in mind, and upgrade every 3 years. Here are some unexpected yet concrete recommendations:

- **Samsung Galaxy A54**: Offers a great balance of performance and battery life, with a 5000mAh battery that can last over a day.
- **Google Pixel 6a**: Known for its excellent camera and software experience, it also has a decent battery life and is often available at a lower price.
- **OnePlus Nord 2T**: This phone features fast charging and a solid battery performance, making it a great option for heavy users.

Tip: Consider looking for deals or refurbished models to stretch your budget further. Websites like [Swappa](#) or [Back Market](#) can offer great savings on quality devices.

Type a message... Submit

(a) Conventional text-based interface (Text-CQ)

Recommend a smartphone.

(A) What is your budget for a new smartphone? (in dollars) (B) 700

Do you prefer iOS or Android operating systems?
☐ iOS ☒ Android

What features are most important to you?
☐ Camera ☒ Battery ☐ Storage ☐ Performance ☐ Design

Are you looking for a specific brand?
 No

How often do you upgrade your smartphone?
 Every 3 years

(C) Submit

(D) {"budget":"700","os-preference":"Android","features":"battery","brand":"No","upgrade-frequency":"every 3 years"}

(E) You have a budget of \$700, prefer an Android smartphone, prioritize battery life, and plan to upgrade every 3 years. Here are some unexpected yet concrete recommendations:

- **Samsung Galaxy A54**. Known for its excellent battery life and solid performance, it fits well within your budget.

Type a message... (F) Submit

(b) Proposed GUI-based interface (GenerativeGUI)

Figure 1: Examples of the user interface ($N_q = 5$): (A) User's input text. (B) System's response, including CQs and the generated GUI. (C) Submit button that is attached to the system's response with CQs and GUIs. (D) User's response via the GUI, automatically converted into a JSON string. (E) System's final response. (F) Interface for the user's text input.

LLM generates GUI code (HTML) at each turn of the conversation. The generated code is immediately rendered and displayed to the user (Fig. 1b). This method is particularly effective for LLM-based general-purpose CAI systems, as it allows for real-time GUI generation that adapts to the evolving conversational context.

The proposed GenerativeGUI poses two challenges. The first challenge is whether the generated GUIs can improve CAI usability. While existing LLMs have demonstrated strong performance in generating program code [14, 28], their ability to generate appropriate GUI code for user interaction remains untested. The second challenge is related to the impact of system response time on usability of a CAI. Since the GenerativeGUI may increase response time due to longer output tokens including HTML tags, we must verify the impact of this on the user experience.

This study evaluates GenerativeGUI, focusing on requirement clarification interaction. In LLM-based CAI systems, users often struggle to specify their requirements, leading to responses that don't meet their needs [16, 23, 31]. While previous approaches use clarifying questions (CQs) [7, 18, 21, 33], they may increase user effort. GenerativeGUI reduces this effort by providing interactive widgets like radio buttons and calendars, enabling easier input from users. We design recommendation tasks with underspecified requirements in the prompts, which makes them challenging to address within a single conversational turn for the evaluation. Additionally, since it's not always possible to estimate all the necessary CQs for task completion, we evaluate the system under conditions of both sufficient and limited CQs.

The goal of this study is to evaluate whether GenerativeGUI enhances usability in the context of requirement clarification interactions. Specifically, usability is evaluated based on effectiveness, efficiency (in terms of cognitive load and time), and satisfaction. In this study, we answer two research questions:

RQ1 : Does GenerativeGUI improve usability metrics such as effectiveness, cognitive load, time efficiency, and user satisfaction?

RQ2 : What is the relationship between the number of clarifying questions generated by GenerativeGUI, response duration, and usability?

To address **RQ1**, we present users with different CAI systems and compare perceived usability. Additionally, to address **RQ2**, we investigate how different numbers of clarifying questions impact response time and usability.

2 Related Works

LLM and GUI. Recent multimodal LLMs are enhancing the ability to understand and interact with GUIs [3], with applications such as webpage comprehension [13, 19] and GUI-based navigation [22, 29, 32]. However, research on generating GUIs for user interactions with LLMs is limited. Graphologue [15], for example, converts LLM responses into interactive diagrams for better information exploration. ExploreLLM [20] helps structure tasks and guide users through complex choices, illustrating the potential of combining natural language and graphical interfaces. A commonality among these previous studies is that, although they generate

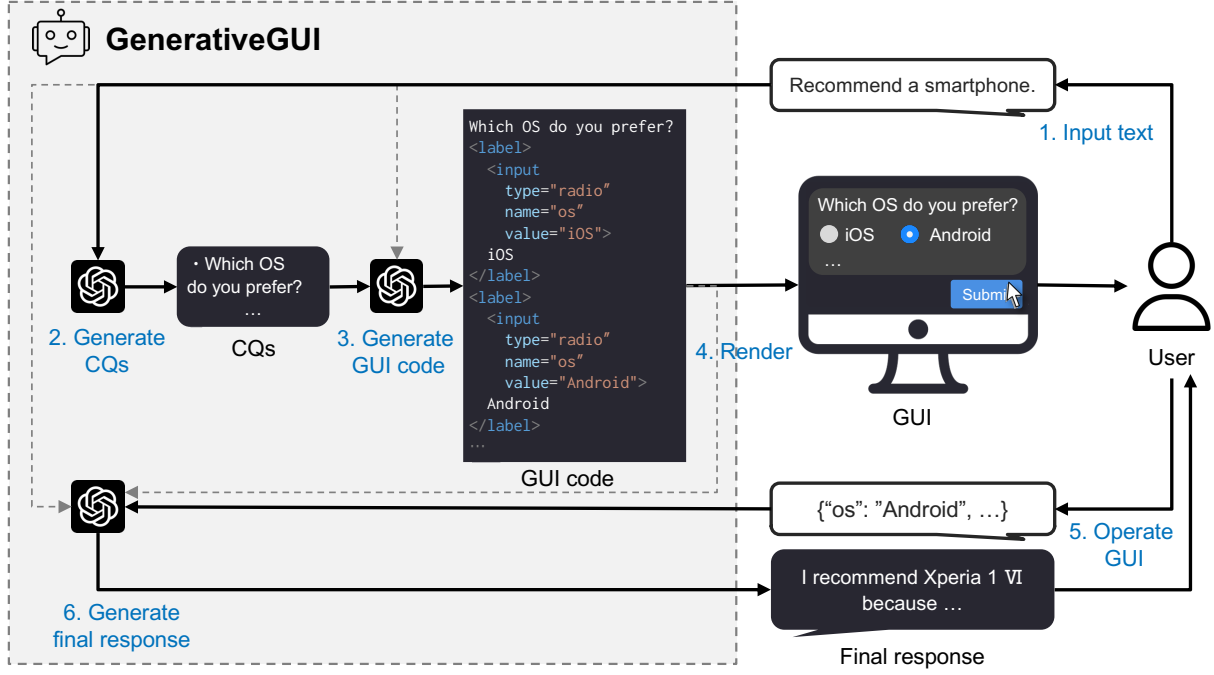


Figure 2: Schematic diagram of the proposed GenerativeGUI.

displayed content using LLMs, the GUI elements employed are implemented in advance, limiting the scope of possible tasks. Designing a flexible, context-sensitive GUI for diverse user interactions remains a significant challenge.

Clarifying Questions. To address ambiguous and multifaceted instructions from users, previous studies have explored CQ generation for information retrieval [2, 30, 35], conversational recommendation systems [26, 36], and LLM-based assistants [5, 8, 18, 23, 33]. While CQs help specify the user’s requirements and thus facilitate meeting their needs, they also increase the number of interaction turns, requiring users to input large amounts of text and thereby reducing efficiency. This study investigates whether usability can be improved by using GenerativeGUI in user responses to CQs.

Voice Assistants. Adopting a voice user interface (VUI) such as Amazon Alexa, Google Assistant, or Siri is a common CAI design for improving usability [9, 37]. VUI can facilitate human–computer interactions in a natural and intuitive way, similar to the conversations between human beings [37]. On the other hand, there are security and privacy concerns with VUI, such as unauthorized access to personal data and eavesdropping through always-on microphones [4, 27]. This study focuses on CAI based on the visual channel, which does not involve such security or privacy concerns.

3 Methods

3.1 The GenerativeGUI System

Figure 2 shows the schematic diagram of the proposed system. First, the user sends a message describing the task via the text input field. Then, an LLM generates CQs to clarify the user’s requirements.

The LLM is instructed to generate N_q CQs in a single inference using a one-shot example (Appendix A.1), where N_q represents the number of generated CQs.

Next, the LLM generates the GUI code (HTML) to allow the user to easily answer the generated CQs. The LLM is instructed to use appropriate input elements such as calendar, checkbox, radio button, and range, depending on the question, using a one-shot example (Appendix A.2). Each element is assigned a name and value, which are used to convert user responses using the GUI into text. The generated GUI code is included as utterance text in the conversation context and is considered in subsequent system turns. Although simultaneously generating CQs and the GUI code in a single inference can reduce the system’s response time, we chose sequential inferences. This allowed us to evaluate GenerativeGUI and conventional systems based on the same CQs, as described in Sec. 3.2.2.

The generated GUI is rendered and displayed to the user as the system’s utterance on the chat interface. When the user interacts with the GUI and clicks the submit button, the form’s data is converted to a JSON string (e.g., `{“os”: “Android”, “price”: “700”}`) based on the name and value of each element. The converted string is then used as the user’s utterance text for that turn. This simple format is sufficient for the system to understand the user’s intent because the conversational context contains the generated GUI code, including CQ text as well as the name and value of each GUI element. Finally, the system generates a final response in text by considering the conversational context (Appendix A.3).

3.2 Evaluation Protocol

We conducted an online between-subject experiment to evaluate the usability of different CAI systems. To address **RQ1**, the proposed system was compared with two conventional text-based CAIs: the baseline without CQs (Text-NoCQ) and the one that asks CQs (Text-CQ). For **RQ2**, we evaluated the Text-CQ and GenerativeGUI systems with $N_q = 2$ and $N_q = 5$. $N_q = 5$ represents a scenario with a sufficient number of CQs, while $N_q = 2$ assumes that only a limited number of CQs are available.

We asked the participants to perform the following: (1) Complete a demographic survey. (2) Select an experimental task from a presented list. (3) Interact with a CAI system. (4) Answer a post-interaction questionnaire. To compare the average task completion time for each system, we prohibited follow-up questions from users, and instruct them to evaluate the interaction up to the point when the system generated its answer to the task. The participants were instructed to use a PC to complete a task.

3.2.1 Text-NoCQ. This system mimics typical interactions of a conventional text-based CAI, in other words, it generates neither a CQ nor a GUI. As a result, there is only a single-turn interaction between the user and the system in the experimental setup. The system prompt is consistent with the one for final response generation in the proposed system. This system may provide responses that are not beneficial to the user if the prompt entered by the user does not fully express their requirements [17].

3.2.2 Text-CQ. This system is designed to mimic a conventional text-based CAI that generates CQs for requirement clarification (Fig. 1a). It generates CQs but not a GUI. All interactions including the CQ display and user responses are text-based. There are two-turn interactions between the user and the system in the experimental setup. The system prompts for CQ and final response generation are consistent with the proposed system. The system may provide inefficient interaction because users input answers to the CQs with the text-based interface.

3.2.3 Implementation. We implemented the CAI systems using HTML, CSS, and JavaScript for the frontend, with OpenAI GPT-4o-mini (gpt-4o-mini-2024-07-18) API and Flask framework for the backend. Note that the interaction design is agnostic to the LLM API of choice, as long as the LLM has sufficient instruction following and reasoning capabilities. The systems did not show the incremental generation process like the ChatGPT web interface and displayed a spinner while waiting for the system's reply. In the text-based input interface, we enabled message submission using the Enter key as a shortcut to evaluate usability under typical CUI conditions.

3.3 Task

Previous researches investigated requirement clarification interaction in programming [21, 31] and recommendation [24, 34] tasks. This study chose recommendation tasks for the experiments because they do not require specialized expertise for evaluation. Since different experimental tasks can influence the user's interaction, we designed the following seven distinct tasks: "Recommend a smartphone," "Recommend a book," "Help plan a trip," "Suggest ways to spend a day off," "Provide an English learning plan," "Give tips for

managing a budget," and "Help make a restaurant reservation." To motivate the interaction with a CAI system, the participants were guided to select a task with which they felt most comfortable, thus avoiding any task assignment biases [12].

3.4 Measurements

From the pre-task questionnaire, we gathered demographic information, including their gender and age. We evaluated the system's usability, based on the post-task questionnaire and the chatbot's interaction logs, which record the time each turn was generated. Usability was evaluated based on ISO/IEC 25010 [10], with respect to effectiveness, efficiency (in terms of cognitive load and time), and satisfaction. All of the questionnaire was based on 7-point Likert scale (Appendix B).

Effectiveness was evaluated based on whether the system's response (1) meets the user's specific needs and context, (2) feels personalized, and (3) is practical and useful when applied. The average of these three aspects was then used to calculate the effectiveness score. **Cognitive Load Efficiency (CLE)** was assessed using the NASA-TLX [11], which consists of six questions. After correcting for the reverse-coded items, the average of each question was used to compute the scores. Additionally, the average of all six questions was defined as the CLE score. **Satisfaction** score was determined based on the user's response to a single question about their satisfaction with using the system. **Task completion time** was defined as the time from the user's initial input to the generation of the system's final response, and was calculated from the interaction logs. **CQ and GUI Validity** scores were also evaluated to preliminarily evaluate the validity of the generated CQs and GUIs, which are omitted in the main text (Appendix C). We also confirmed that all the generated HTML code was parsable and free from syntactic, semantic, or structural errors.

3.5 Participants

We recruited crowd workers from Amazon Mechanical Turk [6]. They had received qualifications for consistent performance [1]. They were native English-speaking participants from Australia, Canada, New Zealand, the UK, or the US and compensated at the rate of \$13 per hour. They were in the habit of using generative AI, such as ChatGPT or Gemini, at least once a month. A total of 500 participants were recruited, with 100 participants assigned to each of the five conditions. To ensure the data quality, we excluded the responses from workers who provided monotonous annotations. As a result, data from 469 participants ($N=93, 94, 94, 94$, and 94 for Text-NoCQ, Text-CQ ($N_q = 2$), Text-CQ ($N_q = 5$), GenerativeGUI ($N_q = 2$), and GenerativeGUI ($N_q = 5$), respectively) were used for the analysis. The gender distribution consisted of 191 females, 270 males, and 8 non-binary individuals. In terms of age, 27 participants were 18–24 years old, 241 were 25–34, 155 were 35–44, 30 were 45–54, and 16 were older than 54.

4 Results

To compare three chatbot systems, Text-NoCQ, Text-CQ, and GenerativeGUI, we conducted the analysis of variance (ANOVA), assuming the independence of different groups. When ANOVA showed

Table 1: Evaluation Results. $x(y)$ represents a value with a mean of x and a standard deviation of y . *, †, and § denote significant differences with Text-NoCQ, Text-CQ, and GenerativeGUI, respectively, in the Tukey method-based post-hoc analysis (* / † / § p-value < 0.05; ** / †† / §§ p-value < 0.01).

Metrics	$N_q = 2$			$N_q = 5$		
	Text-NoCQ(*)	Text-CQ(†)	GenerativeGUI(§)	Text-NoCQ(*)	Text-CQ(†)	GenerativeGUI(§)
Usefulness (↑)	4.47 (1.92)	5.00 (1.88)§§	4.14 (1.95)††	4.47 (1.92)§	5.02 (1.52)	5.22 (1.77)*
Needs met (↑)	4.39 (2.05)	5.05 (1.96)§§	4.14 (1.92)††	4.39 (2.05)§§	4.56 (1.74)§	5.22 (1.93)**†
Personalization (↑)	3.46 (2.01)	4.13 (1.98)§	3.45 (1.93)†	3.46 (2.01)†§	4.29 (1.67)*	4.21 (2.32)*
Effectiveness (↑)	4.11 (1.90)†	4.73 (1.80)*§§	3.95 (1.86)††	4.11 (1.90)§	4.62 (1.58)	4.89 (1.97)*
Mental demand (↑)	5.19 (1.09)	4.82 (1.68)	5.19 (1.13)	5.19 (1.09)††§	4.08 (1.65)**§	4.62 (1.51)*†
Physical demand (↑)	5.67 (0.65)	5.53 (1.28)	5.83 (0.79)	5.67 (0.65)††§§	5.30 (0.50)**	5.19 (0.91)**
Temporal demand (↑)	5.08 (1.59)	4.54 (2.26)	5.06 (1.47)	5.08 (1.59)	4.74 (1.31)	4.71 (1.60)
Performance (↑)	3.70 (1.87)	4.21 (2.03)	3.85 (1.84)	3.70 (1.87)§§	3.79 (1.53)§	4.47 (1.66)**†
Effort (↑)	5.39 (0.68)††	4.67 (1.71)**§	5.19 (1.62)†	5.39 (0.68)††§	4.80 (1.34)**	5.03 (0.77)*
Frustration (↑)	5.23 (1.36)	4.96 (1.96)	5.25 (1.41)	5.23 (1.36)	4.82 (1.19)	4.81 (1.79)
CLE (↑)	5.03 (0.61)†	4.77 (0.89)*§	5.04 (0.58)†	5.03 (0.61)††§	4.58 (0.39)**§	4.80 (0.75)*†
Satisfaction (↑)	4.09 (1.97)††	4.90 (1.93)**§§	3.91 (1.90)††	4.09 (1.97)§§	4.38 (1.78)§§	5.20 (1.73)**††
Task completion time (seconds) (↓)	4.09 (0.54)††§§	32.54 (18.08)**	36.63 (19.38)**	4.09 (0.54)††§§	73.87 (29.85)**§§	57.54 (33.82)**††

significance, we performed Post-Hoc pair-wise comparisons between conditions based on the Tukey method.

4.1 RQ1: Does GenerativeGUI improve usability metrics such as effectiveness, cognitive load, time efficiency, and user satisfaction?

Table 1 shows the evaluation results. We first compare the three conditions: Text-NoCQ, Text-CQ ($N_q=5$), and GenerativeGUI ($N_q=5$), assuming a sufficient number of CQs are estimated from the context.

Effectiveness: Text-CQ was evaluated as statistically significantly more personalized compared to Text-NoCQ, which led to a trend of improvement in the effectiveness score. In contrast, GenerativeGUI improved effectiveness compared to both Text-NoCQ and Text-CQ, particularly in the 'Needs met' item. These results suggest that CQs enhance effectiveness, and generating a GUI further improves it. To explore the reasons behind the differences between Text-CQ and GenerativeGUI, we briefly analyzed conversation logs. In 17 out of 94 Text-CQ sessions, participants did not respond to all five CQs, resulting in lower effectiveness scores ($M=3.35$) compared to other sessions ($M=4.90$; $p<0.01$). In five of these sessions, they mentioned accidentally sending messages due to the Enter key shortcut, which led to incomplete responses. No such issues were observed with the GUI, where users consistently responded to multiple CQs, thereby maintaining effectiveness. The result demonstrates that long text input can occasionally induce operational errors, negatively affecting the interaction quality, depending on the system's implementation.

Cognitive load efficiency (CLE): Text-CQ increased cognitive load in 'Mental demand,' 'Physical demand,' and 'Effort' compared to Text-NoCQ, while GenerativeGUI reduced mental demand and outperformed Text-CQ in 'Performance.' The reduction in mental demand by GenerativeGUI is likely due to the intuitive and visual elements of the GUI, which aid the user's understanding of the

presented information. Although the GUI still requires screen interactions through tools like a mouse, which means physical demand is not improved. These findings indicate that responding to CQs via text interface are mentally and physically demanding and require more effort, but the GUI alleviates mental load. The improvement in performance can be explained by the same reasons as effectiveness.

Satisfaction: GenerativeGUI significantly improve satisfaction compared to both the Text-NoCQ and Text-CQ systems. These results suggest that asking CQs alone does not improve CAI satisfaction, whereas generating a GUI leads to a noticeable improvement.

Task completion time: Text-CQ significantly increases task completion time compared to Text-NoCQ, while GenerativeGUI reduces it compared to Text-CQ. These results imply that CQs prolong task completion due to more conversation turns, while generating a GUI helps reduce this increase.

Summary: Generating CQs improved effectiveness but also increased cognitive load and task completion time due to additional conversation turns. This trade-off was mitigated by the use of the GUI, which reduced mental demand, required effort and task completion time, which contributed to improvements in satisfaction. There results indicate that GenerativeGUI can improve usability compared to conventional text-based CUI systems in requirement clarification interactions.

4.2 RQ2: What is the relationship between the number of clarifying questions generated by GenerativeGUI, response duration, and usability?

To address RQ2, we compared three conditions: Text-NoCQ, Text-CQ ($N_q=2$), and GenerativeGUI ($N_q=2$), assuming a limited number of CQs are estimated from the context. We found that while the CLE of GenerativeGUI improved, its effectiveness, satisfaction, and task completion time were worse compared to Text-CQ. GenerativeGUI

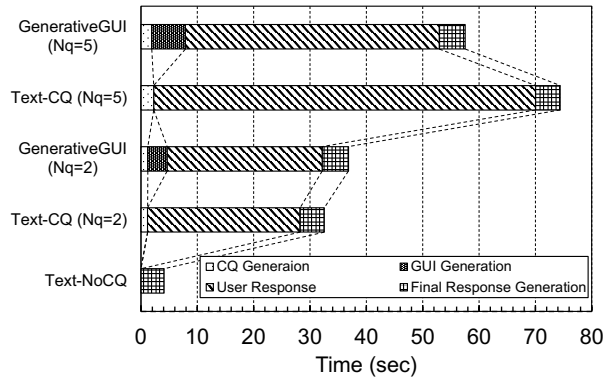


Figure 3: Breakdown of the task completion time.

did not improve usability when the number of generated CQs was limited.

To explore the reasons behind the difference in effectiveness, we analyzed conversation logs. Among 94 participants, 27 who provided additional information in CQs had higher effectiveness scores ($M=5.63$) compared to the 67 who did not ($M=4.36$; $p<0.01$). For example, when asked about their travel budget, a participant provided a detailed breakdown including transportation, accommodation, and total costs, while GenerativeGUI limited input to a single numerical value, hindering task completion. With fewer CQs, the text interface showed an advantage in allowing users to proactively offer more useful information.

We also analyzed the increased task completion time with GenerativeGUI (Fig. 3). Under the $N_q=5$ condition, GenerativeGUI required additional 6.0 seconds for GUI generation but reduced user response time by 22.7 seconds compared Text-CQ. However, with $N_q=2$, the GUI generation took 3.5 seconds and did not reduce response time. These results suggest that with many CQs, the GUI reduces effort, but with fewer CQs, the added time outweighs its benefits.

4.3 RQ3: Which is more usable: a text interface with fewer CQs or a GenerativeGUI with more CQs?"

In the previous sections, we observed opposing trends: Text-CQ showed higher usability with fewer CQs, while GenerativeGUI performed better with more CQs. Therefore, we added an additional research question, **RQ3**, to compare the usability of a text interface with fewer CQs (Text-CQ, $N_q = 2$) and a GenerativeGUI with more CQs (GenerativeGUI, $N_q = 5$). Forty participants interacted with both systems in a randomized order, performing the same task and then selecting the superior system based on effectiveness, cognitive load efficiency, and user satisfaction. The results showed that for effectiveness, satisfaction, and cognitive load efficiency (temporal demand and performance), over 70% of participants judged GenerativeGUI to be superior, with significant differences observed. For the remaining items, the two systems performed equally. These results suggest that GenerativeGUI, by allowing more CQs to be

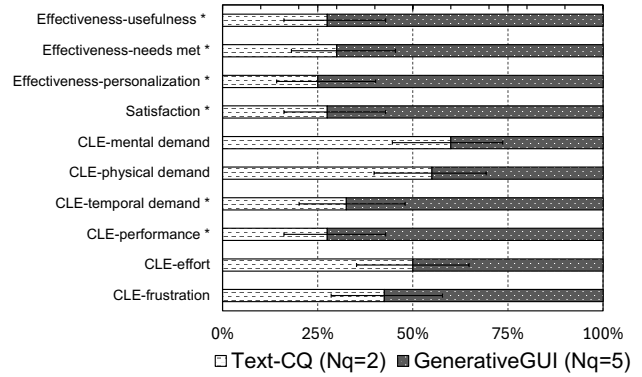


Figure 4: Average preference scores with 95% confidence intervals. * denotes significant differences ($p < 0.01$).

asked, offers a more efficient and satisfying interface compared to a text-based system with fewer CQs.

5 Discussion, Limitations and Future Work

This paper proposed GenerativeGUI, an approach where LLMs dynamically generate GUIs tailored to the ongoing conversation for more efficient user interaction. The user study found that while generating CQs improved effectiveness, it also increased mental and physical demand, required effort, and task completion time due to additional conversation turns. The GUI mitigated this trade-off by reducing mental demand and task completion time. When the number of available CQs was limited, the text interface demonstrated higher usability due to the advantage of users actively providing additional information. In contrast, the GUI, which prompted direct responses to CQs, led to a decrease in effectiveness and satisfaction.

This study primarily focused on demonstrating that GenerativeGUI can improve usability; the evaluation of the validity of the GUI generated by the LLM was preliminary. Future work should aim to create a benchmark, and compare the ability of various LLMs to generate appropriate GUIs. The experiments were based on a fixed number of interaction turns, assuming CQ and GUI generation were required. However, in real-world use cases, it is necessary to determine both the number of interaction turns and whether CQ and GUI generation are needed, considering the context. Future evaluations should better simulate such real-world conditions. The GUI generation method in this study was dependent on the one-shot example, which limits its generalizability. Future work should involve using techniques such as supervised fine-tuning to develop an LLM capable of generating appropriate GUI code for a wider range of tasks without specific in-context examples.

References

- [1] 2005–2018. Amazon Mechanical Turk. <https://www.mturk.com/worker/help>.
- [2] Mohammad Aliannejadi, Hamed Zamani, Fabio Crestani, and W Bruce Croft. 2019. Asking clarifying questions in open-domain information-seeking conversations. In *Proceedings of the 42nd international acm sigir conference on research and development in information retrieval*. 475–484.
- [3] Dongping Chen, Yue Huang, Siyuan Wu, Jingyu Tang, Liuyi Chen, Yilin Bai, Zhigang He, Chenlong Wang, Huichi Zhou, Yiqiang Li, et al. 2024. GUI-WORLD: A Dataset for GUI-oriented Multimodal LLM-based Agents. *arXiv preprint arXiv:2406.10819* (2024).

- [4] Peng Cheng and Utz Roedig. 2022. Personal voice assistant security and privacy—a survey. *Proc. IEEE* 110, 4 (2022), 476–507.
- [5] Jeremy R Cole, Michael JQ Zhang, Daniel Gillick, Julian Martin Eisenschlos, Bhuwan Dhingra, and Jacob Eisenstein. 2023. Selectively answering ambiguous questions. *arXiv preprint arXiv:2305.14613* (2023).
- [6] Matthew JC Crump, John V McDonnell, and Todd M Gureckis. 2013. Evaluating Amazon’s Mechanical Turk as a tool for experimental behavioral research. *PloS one* 8, 3 (2013), e57410.
- [7] Yang Deng, Wenqiang Lei, Minlie Huang, and Tat-Seng Chua. 2023. Rethinking Conversational Agents in the Era of LLMs: Proactivity, Non-collaborativity, and Beyond. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 298–301.
- [8] Yang Deng, Lizi Liao, Liang Chen, Hongru Wang, Wenqiang Lei, and Tat-Seng Chua. 2023. Prompting and evaluating large language models for proactive dialogues: Clarification, target-guided, and non-collaboration. *arXiv preprint arXiv:2305.13626* (2023).
- [9] Faruk Lawal Ibrahim Dutsinma, Debajyoti Pal, Suree Funilkul, and Jonathan H Chan. 2022. A systematic review of voice assistant usability: An ISO 9241–11 approach. *SN computer science* 3, 4 (2022), 267.
- [10] International Organization for Standardization. 2011. *ISO/IEC 25010: 2011: Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—System and Software Quality Models*. ISO/IEC.
- [11] Sandra G Hart. 2006. NASA-task load index (NASA-TLX); 20 years later. In *Proceedings of the human factors and ergonomics society annual meeting*, Vol. 50. Sage publications Sage CA: Los Angeles, CA, 904–908.
- [12] Chien-Ju Ho and Jennifer Vaughan. 2012. Online task assignment in crowdsourcing markets. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 26. 45–51.
- [13] Wenyi Hong, Wei Han Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2024. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14281–14290.
- [14] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. *arXiv preprint arXiv:2406.00515* (2024).
- [15] Peiling Jiang, Jude Rayan, Steven P Dow, and Haijun Xia. 2023. Graphologue: Exploring large language model responses with interactive diagrams. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–20.
- [16] Gangwoo Kim, Sungdong Kim, Byeongguk Jeon, Joonsuk Park, and Jaewoo Kang. 2023. Tree of Clarifications: Answering Ambiguous Questions with Retrieval-Augmented Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 996–1009.
- [17] Yoonsu Kim, Jueon Lee, Seoyoung Kim, Jaehyuk Park, and Juho Kim. 2024. Understanding users’ dissatisfaction with chatgpt responses: Types, resolving tactics, and the effect of knowledge level. In *Proceedings of the 29th International Conference on Intelligent User Interfaces*. 385–404.
- [18] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2022. Clam: Selective clarification for ambiguous questions with generative language models. *arXiv preprint arXiv:2212.07769* (2022).
- [19] Hanyu Lai, Xiao Liu, Lat Long Iong, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, et al. 2024. AutoWebGLM: Bootstrap And Reinforce A Large Language Model-based Web Navigating Agent. *arXiv preprint arXiv:2404.03648* (2024).
- [20] Xiao Ma, Swaroop Mishra, Ariel Liu, Sophie Ying Su, Jilin Chen, Chinmay Kulkarni, Heng-Tze Cheng, Quoc Le, and Ed Chi. 2024. Beyond chatbots: Explorellm for structured thoughts and personalized model responses. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–12.
- [21] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binqian Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. 2023. ClarifyGPT: Empowering LLM-based Code Generation with Intention Clarification. *arXiv preprint arXiv:2310.10996* (2023).
- [22] Runliang Niu, Jindong Li, Shiqi Wang, Yali Fu, Xiyu Hu, Xueyuan Leng, He Kong, Yi Chang, and Qi Wang. 2024. Screenagent: A vision language model-driven computer control agent. *arXiv preprint arXiv:2402.07945* (2024).
- [23] Cheng Qian, Bingxiang He, Zhong Zhuang, Jia Deng, Yujia Qin, Xin Cong, Zhong Zhang, Jie Zhou, Yankai Lin, Zhiyuan Liu, et al. 2024. Tell me more! towards implicit user intention understanding of language model driven agents. *arXiv preprint arXiv:2402.09205* (2024).
- [24] Xuhui Ren, Hongzhi Yin, Tong Chen, Hao Wang, Zi Huang, and Kai Zheng. 2021. Learning to ask appropriate questions in conversational recommendation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 808–817.
- [25] Steven I Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D Weisz. 2023. The programmer’s assistant: Conversational interaction with a large language model for software development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. 491–514.
- [26] Anna Sepiarskaia, Julia Kiseleva, Filip Radlinski, and Maarten de Rijke. 2018. Preference elicitation as an optimization problem. In *Proceedings of the 12th ACM Conference on Recommender Systems*. 172–180.
- [27] M Vimalakumar, Sujeet Kumar Sharma, Jang Bahadur Singh, and Yogesh K Dwivedi. 2021. ‘Okay google, what about my privacy?’: User’s privacy perceptions and acceptance of voice based digital assistants. *Computers in Human Behavior* 120 (2021), 106763.
- [28] Jianxun Wang and Yixiang Chen. 2023. A Review on Code Generation with LLMs: Application and Evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*. IEEE, 284–289.
- [29] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158* (2024).
- [30] Hamed Zamani, Susan Dumais, Nick Craswell, Paul Bennett, and Gord Lueck. 2020. Generating clarifying questions for information retrieval. In *Proceedings of the web conference 2020*. 418–428.
- [31] JD Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny can’t prompt: how non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–21.
- [32] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2023. Appagent: Multimodal agents as smartphone users. *arXiv preprint arXiv:2312.13771* (2023).
- [33] Tong Zhang, Peixin Qin, Yang Deng, Chen Huang, Wenqiang Lei, Junhong Liu, Dingnan Jin, Hongru Liang, and Tat-Seng Chua. 2024. CLAMBER: A Benchmark of Identifying and Clarifying Ambiguous Information Needs in Large Language Models. *arXiv preprint arXiv:2405.12063* (2024).
- [34] Jie Zou. 2022. Improving search and recommendation by asking clarifying questions. In *ACM SIGIR Forum*, Vol. 55. ACM New York, NY, USA, 1–2.
- [35] Jie Zou, Mohammad Aliannejadi, Evangelos Kanoulas, Maria Soledad Pera, and Yiqun Liu. 2023. Users meet clarifying questions: Toward a better understanding of user interactions for search clarification. *ACM Transactions on Information Systems* 41, 1 (2023), 1–25.
- [36] Jie Zou, Yifan Chen, and Evangelos Kanoulas. 2020. Towards question-based recommender systems. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 881–890.
- [37] Dilawar Shah Zwakman, Debajyoti Pal, and Chonlameth Arpanikanondt. 2021. Usability evaluation of artificial intelligence-based voice assistants: The case of Amazon Alexa. *SN Computer Science* 2, 1 (2021), 28.

A Prompts

A.1 System Prompt for Generating CQs

We apply the following system prompt to generating CQs for Text-CQ and GenerativeGU. The detailed content is as follows:

System prompt for generating CQs:

```
Generate questions to confirm the user's specific
situation in order to provide useful and specific
responses based on the user's input.
Please generate EXACTLY {number_of_questions} questions.

Output in the following format:
- List the questions in a list format.
- Make sure to generate exact three most important
  questions.
- Make sure each question is concise and less than 15
  words.

Below is an example of input and output:
User: Please provide advice on a weight loss plan.
Assistant: ["What is your current weight and target
weight?", "Do you have any dietary restrictions or
medical conditions?"]
```

A.2 System Prompt for Generating GUIs

We apply the following system prompt to generating GUIs for GenerativeGU. The detailed content is as follows:

System prompt for generating GUIs:

```
Generate usable GUI components (e.g., calendars,
checkboxes, radio buttons, ranges) to minimize
cognitive load for user inputs.
Output format:
- JSON with key 'qas', value: list of dicts. Each dict
  has 'question' (string) and 'answer' (HTML string).
- Always specify units for numerical inputs (e.g., 'What
  is your budget? (in dollars)').
- Exclude markdown or formatting, output JSON string only
  .

Example input:
user: "Please give me advice on a weight loss plan."
assistant: ["What is your target duration for weight loss
?", "What is your current exercise routine?", "Do you
have any dietary preferences or restrictions?"].

Example output: {"qas": [{"in-context-example-1"}, {"in-
context-example-2"}, {"in-context-example-3"}]}.
```

in-context-example-1:

```
{
  "question": "<label for='weight-goal'>What is your
target duration for weight loss?</label>",
  "answer": "<input type='range' id='duration' name
='duration' min='1' max='12' value='3'
oninput='document.getElementById('duration-
value').innerText = this.value'><span id='
duration-value' class='slider-value'>3</span>
(months)"
}
```

in-context-example-2:

```
{
  "question": "<label for='exercise-habits'>What is
your current exercise routine?</label>",
```

```
"answer": "<label><input type='radio' name='
exercise-habits' value='none'>I don't
exercise</label><label><input type='radio'
name='exercise-habits' value='light'>Light
exercise (e.g., walking, yoga)</label><label><
input type='radio' name='exercise-habits'
value='moderate'>Moderate exercise (e.g.,
jogging, cycling)</label><label><input type='
radio' name='exercise-habits' value='intense
'>Intense exercise (e.g., gym workouts, running
)</label>"
}
```

in-context-example-3:

```
{
  "question": "<label for='diet-preferences'>Do you
have any dietary preferences or restrictions?</
label>",
  "answer": "<textarea id='diet-preferences' name='
diet-preferences' placeholder='e.g.,
vegetarian, gluten-free, no sugar'></textarea><
label for='lifestyle'>What are your main time
or lifestyle constraints? (e.g., busy work
schedule, night shifts)</label><textarea id='
lifestyle' name='lifestyle' placeholder='
Enter your constraints'></textarea>"
}
```

A.3 System Prompt for Generating Final Response

We apply the following system prompt to generating a final response for baseline, Text-CQ, and GenerativeGU. The detailed content is as follows:

System prompt for generating final response:

```
Respond with an emphasis on the unexpectedness and
concreteness of the information, so that the
response will be useful in a way that the user has
not considered before.

Please output in the following format.
- The entire final response should not exceed 200 words.
- Use markdown notation such as bullet points and bold
  text.
- Ensure that you use 4 spaces (not 2) for Markdown
  indentation.
- If possible, include external links to provide more
  useful information.
```

B Measurements

The following questionnaires and options were used in the experiments.

CQ and GUI Validity:

- (CQ Validity) How valid were the questions generated by the chatbot for the system to provide a useful response about the topic?
(1) Very Poor (2) Poor (3) Slightly Poor (4) Neutral (5) Slightly Good (6) Good (7) Very Good
- (GUI Validity) How valid were the GUI elements generated by the chatbot (e.g., textarea, dropdown) in supporting your response?
(1) Very Poor (2) Poor (3) Slightly Poor (4) Neutral (5) Slightly Good (6) Good (7) Very Good

Effectiveness:

Table 2: Number of generated GUI elements in 94 sessions by GenerativeGUI ($N_q = 5$).

Tag name (type)	input (radio)	input (number)	select	input (text)	textarea	input (date)	input (checkbox)	input (range)	input (time)
#Elements	105	92	87	76	70	18	16	4	2
#Sessions with Element	77	66	59	76	62	18	16	4	2
Ratio of Sessions with Element	0.82	0.70	0.63	0.81	0.66	0.19	0.17	0.04	0.02

Table 3: Evaluation Results. $x(y)$ represents a value with a mean of x and a standard deviation of y .

Metrics	$N_q = 5$	
	Text-CQ	GenerativeGUI
CQ Validity ($\uparrow$)	5.55 (1.67)	5.82 (1.08)
GUI Validity ($\uparrow$)	-	5.84 (1.33)

- (Effectiveness-usefulness) How useful was the system’s final response when applied in practice?
(1) Very Poor (2) Poor (3) Slightly Poor (4) Neutral (5) Slightly Good (6) Good (7) Very Good
- (Effectiveness-needs met) To what extent did the system’s final response meet your specific needs and situation?
(1) Very Poor (2) Poor (3) Slightly Poor (4) Neutral (5) Slightly Good (6) Good (7) Very Good
- (Effectiveness-personalization) How personalized did the system’s final response feel to you?
(1) Very Poor (2) Poor (3) Slightly Poor (4) Neutral (5) Slightly Good (6) Good (7) Very Good

Cognitive load efficiency (NASA-TLX):

*: Reverse coded items to calculate cognitive load efficiency score in our experiments.

- (*CLE-mental demand) How mentally demanding was the task?
(1) Very Low/Good (2) Low (3) Slightly Low (4) Neutral (5) Slightly High (6) High (7) Very High/Poor
- (*CLE-physical demand) How physically demanding was the task?
(1) Very Low/Good (2) Low (3) Slightly Low (4) Neutral (5) Slightly High (6) High (7) Very High/Poor
- (*CLE-temporal demand) How hurried or rushed was the pace of the task?
(1) Very Low/Good (2) Low (3) Slightly Low (4) Neutral (5) Slightly High (6) High (7) Very High/Poor
- (CLE-performance) How successful were you in accomplishing what you were asked to do?
(1) Very Poor (2) Poor (3) Slightly Poor (4) Neutral (5) Slightly Good (6) Good (7) Very Good
- (*CLE-effort) How hard did you have to work to accomplish your level of performance?
(1) Very Low/Good (2) Low (3) Slightly Low (4) Neutral (5) Slightly High (6) High (7) Very High/Poor
- (*CLE-frustration) How insecure, discouraged, irritated, stressed, and annoyed were you?

(1) Very Low/Good (2) Low (3) Slightly Low (4) Neutral (5) Slightly High (6) High (7) Very High/Poor

Satisfaction:

- (Satisfaction) How satisfied were you with using this system?
(1) Very Low/Good (2) Low (3) Slightly Low (4) Neutral (5) Slightly High (6) High (7) Very High/Poor

C CQ and GUI Validity

We preliminarily confirmed that the CQ and GUI validity were rated with an average score of 5.5 or higher, suggesting they possess sufficient validity, as shown in Tab. 3. The result shows that appropriate CQs and GUIs were generated based on the prompts and in-context samples used in the experiments. Table 2 shows the number of times each GUI element was generated in conversations with GenerativeGUI ($N_q=5$). The system frequently generated elements that support selection from options (input-radio and select) and those for text input (input-number, input-text, and textarea), as well as the calendar (input-date) in 19% of the sessions. The statistical analysis revealed no significant correlation between the number of specific elements and the validity of the GUI, along with other measurements. These suggest that the GenerativeGUI system effectively generates a diverse range of GUI elements, while maintaining a sufficient level of validity.